

Writeup: Crack Me Vault

1. Thông tin thử thách

- **Tên file:** `crack_me_vault.bdsec`
- **Thể loại:** Reverse Engineering
- **Kiến trúc:** ELF 64-bit, x86-64
- **Flag format:** `BDSEC{...}`

Mục tiêu của thử thách là phân tích chương trình, tìm hiểu cơ chế kiểm tra đầu vào và khôi phục flag hợp lệ.

2. Kiểm tra file

Đầu tiên, sử dụng lệnh `file` để xác định định dạng binary:

```
file crack_me_vault.bdsec
```

Kết quả cho thấy đây là một file ELF 64-bit dành cho Linux, được biên dịch dưới dạng PIE và đã bị strip symbol.

Có thể cấp quyền thực thi và chạy thử:

```
chmod +x crack_me_vault.bdsec
./crack_me_vault.bdsec
```

Chương trình yêu cầu nhập flag:

```
Enter the flag:
```

Khi nhập sai, chương trình từ chối quyền truy cập. Điều này cho thấy flag được kiểm tra trực tiếp bên trong binary.

3. Phân tích chuỗi

Sử dụng `strings` để kiểm tra các chuỗi đáng chú ý:

```
strings -a crack_me_vault.bdsec
```

Một số chuỗi quan trọng:

```
Enter the flag:  
[+] Access granted.  
[+] Submit the flag to receive your points.
```

Flag không xuất hiện trực tiếp trong kết quả `strings`, vì vậy chương trình không lưu flag ở dạng plaintext.

4. Phân tích logic chương trình

Khi dịch ngược binary bằng Ghidra hoặc IDA, có thể thấy chương trình không kiểm tra flag bằng một phép so sánh chuỗi thông thường.

Thay vào đó, chương trình triển khai một máy ảo nhỏ, hay còn gọi là custom VM. VM này đọc một chuỗi bytecode và thực hiện các opcode tương ứng.

Các opcode được lưu dưới dạng mã hóa:

```
b4 81 ac 38
```

Sau khi giải mã, bốn opcode thực tế là:

```
0x11  
0x37  
0x6b  
0xe0
```

Ý nghĩa của từng opcode:

Opcode	Chức năng
0x11	Kiểm tra độ dài của input
0x37	Biến đổi từng byte của input
0x6b	So sánh kết quả với mảng byte mục tiêu
0xe0	Kết thúc VM và trả về kết quả

5. Kiểm tra độ dài

Opcode `0x11` kiểm tra input có độ dài chính xác là:

```
50 byte
```

Nếu flag không dài đúng 50 ký tự, chương trình sẽ trả về thất bại ngay lập tức.

Điều kiện đầu tiên:

```
strlen(input) == 50
```

6. Thuật toán biến đổi flag

Với mỗi byte `input[i]`, chương trình thực hiện ba phép toán chính.

Bước 1: XOR với khóa phụ thuộc vị trí

```
value = input[i] ^ ((0x41 + 0x1d * i) & 0xff);
```

Bước 2: Xoay trái byte

Số bit xoay phụ thuộc vào vị trí:

```
value = rol8(value, (i % 7) + 1);
```

Trong đó `rol8` là phép xoay trái trên một byte 8 bit.

Bước 3: Cộng thêm một giá trị phụ thuộc vị trí

```
value = (  
    value + ((0x0b * i) ^ 0x17)  
) & 0xff;
```

Sau đó, kết quả còn được XOR thêm với một giá trị khác trước khi so sánh với mảng đích:

```
encoded = value ^ ((0x44 + 0x0d * i) & 0xff);
```

Như vậy, công thức đầy đủ có thể biểu diễn như sau:

```
encoded[i] =  
    (  
        rol8(  
            input[i] ^ ((0x41 + 0x1d * i) & 0xff),  
            (i % 7) + 1  
        )  
        + ((0x0b * i) ^ 0x17)  
    )  
    ^ ((0x44 + 0x0d * i) & 0xff);
```

7. Mảng byte mục tiêu

Opcode `0x6b` so sánh kết quả biến đổi với một mảng gồm 50 byte nằm trong vùng `.rodata`.

Mảng đích:

```
59 d5 1c 78 61 0f 09 4d c0 bd  
4c 57 e7 67 0a 13 a4 60 7d 27  
0e be 71 69 04 61 9f 66 f1 1a  
31 2c 03 94 e6 a7 6b b7 41 92  
18 4c 59 69 e5 c5 8a 9c 5d b2
```

Do tất cả phép toán đều có thể đảo ngược, ta không cần brute force flag. Chỉ cần thực hiện các phép toán theo thứ tự ngược lại.

8. Đảo ngược thuật toán

Để khôi phục `input[i]` từ `target[i]`, thực hiện:

1. XOR với khóa cuối để loại bỏ lớp XOR.
2. Trừ giá trị đã được cộng.
3. Xoay phải để đảo ngược phép xoay trái.
4. XOR với khóa ban đầu.

Công thức giải mã:

```
expected = target[i] ^ ((0x44 + 0x0d * i) & 0xff)  
  
rotated = (  
    expected ^ ((0x0b * i) ^ 0x17)  
    ^ ((0x44 + 0x0d * i) & 0xff)
```

```
        expected - ((0x0b * i) ^ 0x17)
    ) & 0xff

original = ror8(rotated, (i % 7) + 1)

input_byte = original ^ ((0x41 + 0x1d * i) & 0xff)
```

9. Solver

```
#!/usr/bin/env python3

target = bytes.fromhex(
    "59d51c78610f094dc0bd"
    "4c57e7670a13a4607d27"
    "0ebe716904619f66f11a"
    "312c0394e6a76bb74192"
    "184c5969e5c58a9c5db2"
)

def ror8(value: int, count: int) -> int:
    count %= 8

    return (
        (value >> count)
        | (value << (8 - count))
    ) & 0xff

def solve() -> str:
    flag = bytearray()

    for i, encoded_byte in enumerate(target):
        # Loại bỏ lớp XOR cuối.
        transformed = encoded_byte ^ (
            (0x44 + 0x0d * i) & 0xff
        )

        # Đảo ngược phép cộng.
        rotated = (
            transformed - ((0x0b * i) ^ 0x17)
        ) & 0xff

        # Đảo ngược phép rotate-left.
```

```

        xored = ror8(
            rotated,
            (i % 7) + 1
        )

        # Đảo ngược phép XOR đầu tiên.
        original = xored ^ (
            (0x41 + 0x1d * i) & 0xff
        )

        flag.append(original)

    return flag.decode()

if __name__ == "__main__":
    flag = solve()

    print(f"[+] Flag length: {len(flag)}")
    print(f"[+] Flag: {flag}")

```

Chạy solver:

```
python3 solve.py
```

Kết quả:

```

[+] Flag length: 50
[+] Flag: BDSEC{c0nTr0L_f10w_1s_4_l13_bUt_bYt3c0d3_d03s_n0t}

```

10. Kiểm tra flag

Chạy lại binary và nhập flag vừa khôi phục:

```
./crack_me_vault.bdsec
```

Input:

```
BDSEC{c0nTr0L_f10w_1s_4_l13_bUt_bYt3c0d3_d03s_n0t}
```

Chương trình trả về:

```
Enter the flag:  
[+] Access granted.  
[+] Submit the flag to receive your points.
```

Như vậy flag đã được xác nhận chính xác.

11. Flag

```
BDSEC{c0nTr0L_f10w_1s_4_l13_bUt_bYt3c0d3_d03s_n0t}
```

12. Kết luận

Thử thách sử dụng một custom VM nhỏ để che giấu control flow và thuật toán kiểm tra flag. Tuy nhiên, các phép biến đổi chỉ bao gồm XOR, cộng modulo 256 và rotate bit nên đều có thể đảo ngược hoàn toàn.

Điểm quan trọng khi giải bài:

- Xác định chính xác độ dài flag.
- Hiểu chức năng của từng opcode trong VM.
- Trích xuất đúng mảng byte mục tiêu.
- Đảo ngược các phép toán theo thứ tự ngược lại.
- Kiểm tra flag trực tiếp bằng binary trước khi kết luận.

Tên flag cũng phản ánh ý tưởng chính của bài: control flow có thể gây nhiễu, nhưng bytecode không bảo vệ được dữ liệu nếu thuật toán vẫn có thể bị phân tích và đảo ngược.